



UT-4



Programación de aplicaciones Android Intents



Intents

- Un **Intent** es un objeto de mensajería que podemos usar para solicitar una acción de otro componente de la app.
- Existen tres casos de uso fundamentales:
 - **Iniciar una actividad (Activity)**: este es el caso de uso más frecuente, que permite al usuario navegar entre diferentes pantallas o funcionalidades.
 - **Iniciar o controlar un servicio (Service)**: inicio de tareas que no tienen interfaz de usuario y que deben ejecutarse en segundo plano (incluso cuando la Activity no está visible).
 - **Transmitir un Broadcast (Eventos del Sistema)**: se utilizan para enviar y recibir eventos del sistema o mensajes que deben ser escuchados por uno o más componentes.



Intents Tipos

Existen dos tipos de intents:

- **Intents explícitos:** especifican qué componente de qué aplicación satisfará el **intent**.
- **Intents implícitos:** los intents implícitos no nombran un componente específico, sino que declaran una acción general que se llevará a cabo, lo que permite que un componente de otra app la controle.



Intents Explícitos



- 1) La activity a lanzar se referencia por el **nombre de la clase**.
- 2) Habitualmente la activity está en **la misma app**.
- 3) Se crea un objeto Intent y se inicia con `startActivity()`
- 4) Se pasan datos con `putExtra()`. Los datos son pares clave-valor.

- Ejemplo:

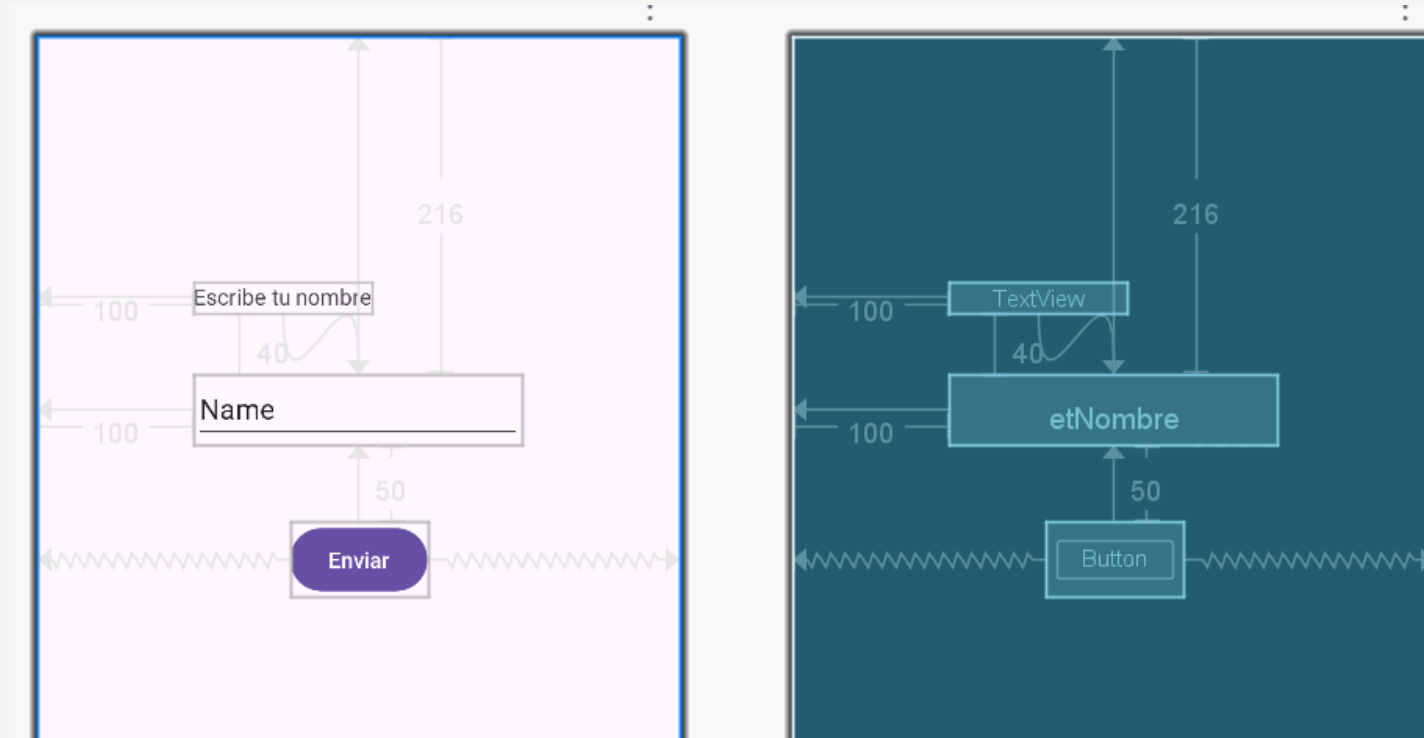
```
val intent:Intent =Intent(this, SecondActivityActividadRespuesta::class.java)

startActivity(intent)
```



Pasar de un Activity a otro (intent explícito)

- Creamos un nuevo proyecto que se llame IntentExplicito
- Creamos el siguiente diseño para el activity principal



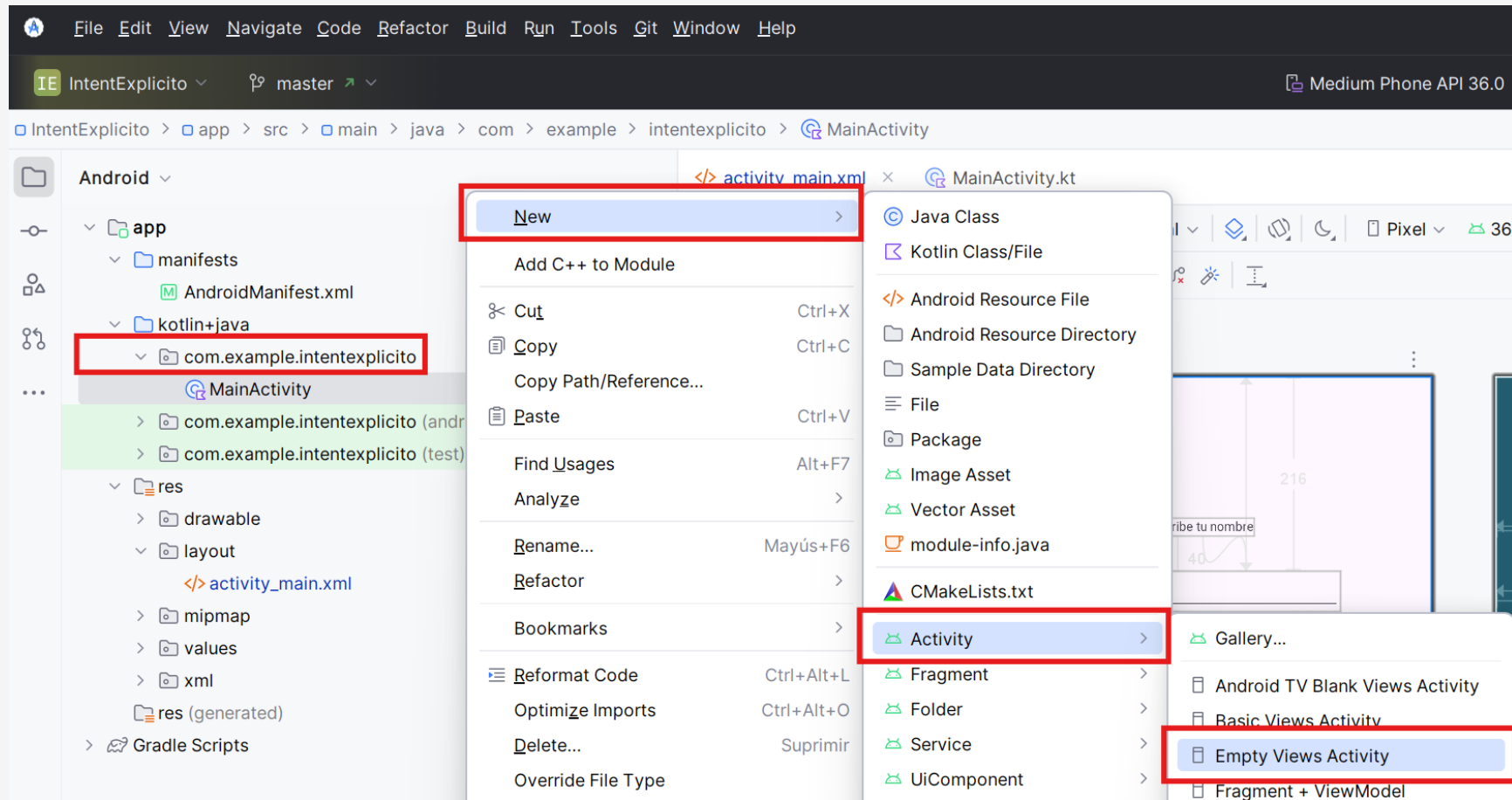
Identificadores:

- **tvNombre**
- **etNombre**
- **btEnviar**



Pasar de un Activity a otro (intent explícito)

- Creamos un segundo Activity: Botón derecho sobre el paquete com.example.intentexplicito





Pasar de un Activity a otro (intent explícito)

- Le llamamos ActivityRespuesta:

New Android Activity

Empty Views Activity

Creates a new empty activity

Activity Name

ActivityRespuesta

☒ Generate a Layout File

Layout Name

activity_respuesta

☐ Launcher Activity

Package name

com.example.intentexplicito

Source Language

Kotlin



Pasar de un Activity a otro (intent explícito)

- Una vez se ha creado el Activity en el archivo **AndroidManifest.xml** se añadirá información de este Activity:

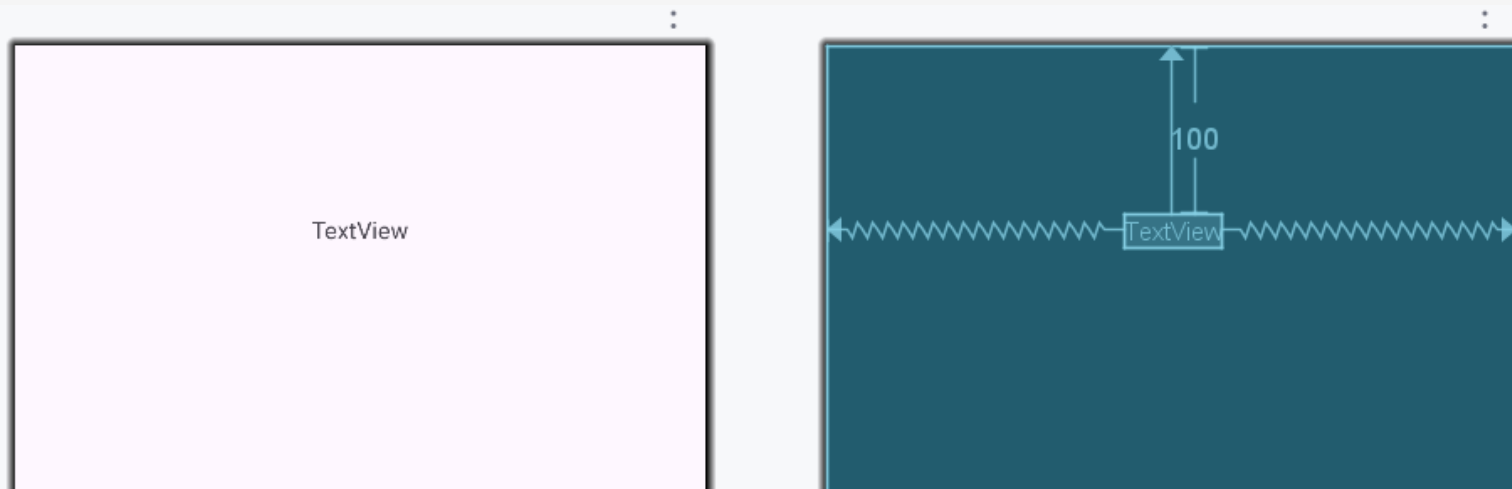
```
<activity  
    android:name=".ActivityRespuesta"  
    android:exported="false" />
```

- **android:name** indica el nombre del Activity
- **android:exported="false"** indica que este Activity es privado y sólo podrá ser abierto por nuestra aplicación. Un valor true permitiría a otras aplicaciones instaladas en el dispositivo llamar o lanzar esta actividad.



Pasar de un Activity a otro (intent explícito)

- El diseño del segundo Activity será el siguiente:



- Identificador del TextView: **tvNombreRecibido**



Pasar de un Activity a otro (intent explícito)

- Añadimos el código para el botón del primer Activity:

```
val btEnviar: Button = findViewById(R.id.btEnviar)

btEnviar.setOnClickListener {

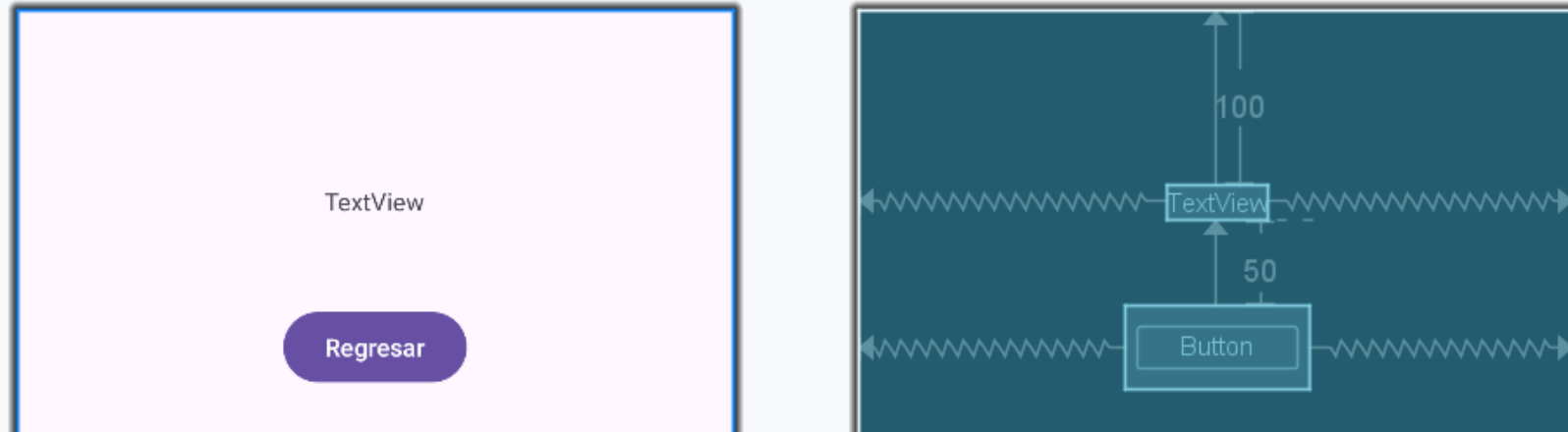
    val intent: Intent = Intent(this, ActivityRespuesta::class.java)

    startActivity(intent)
}
```



Pasar de un Activity a otro (intent explícito)

- Vamos a añadir un botón para volver desde el segundo activity al primero:



- Identificador del botón: **btRegresar**



Pasar de un Activity a otro (intent explícito)

- Añadimos el código para el botón del segundo Activity:

```
val btRegresar: Button = findViewById(R.id.btRegresar)

btRegresar.setOnClickListener {

    finish()

}
```



Pasar de un Activity a otro (intent explícito)

- Vamos a pasar datos desde el primer activity al segundo.
- Usaremos el método **putExtra(String name, String value)** de la clase **Intent**
 - El primer parametro es una etiqueta para poder hacer referencia al valor pasado cuando queramos recuperarlo.
 - El segundo parámetro es el valor que queremos pasar.

```
btEnviar.setOnClickListener {  
    val intent: Intent = Intent(this, ActivityRespuesta::class.java)
```

```
    val etNombre: EditText = findViewById(R.id.etNombre)  
    intent.putExtra("nombre", etNombre.text.toString())
```

```
    startActivity(intent)
```

```
}
```



Pasar de un Activity a otro (intent explícito)

- Recuperamos los datos en el segundo Activity.
- Usaremos el método **getStringExtra(String name)** de la clase **Intent**
 - Recibe un parámetro que es la etiqueta del valor que queremos recuperar.

```
//Obtenemos el intent que comenzó este Activity  
val intent=getIntent()
```

```
//Obtenemos una referencia al textView donde queremos escribir  
val tvNombre: TextView=findViewById(R.id.tvNombreRecibido)
```

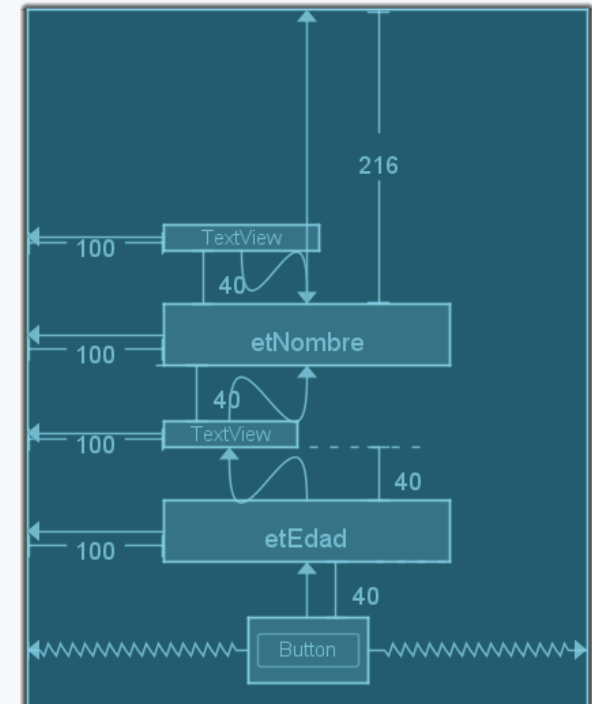
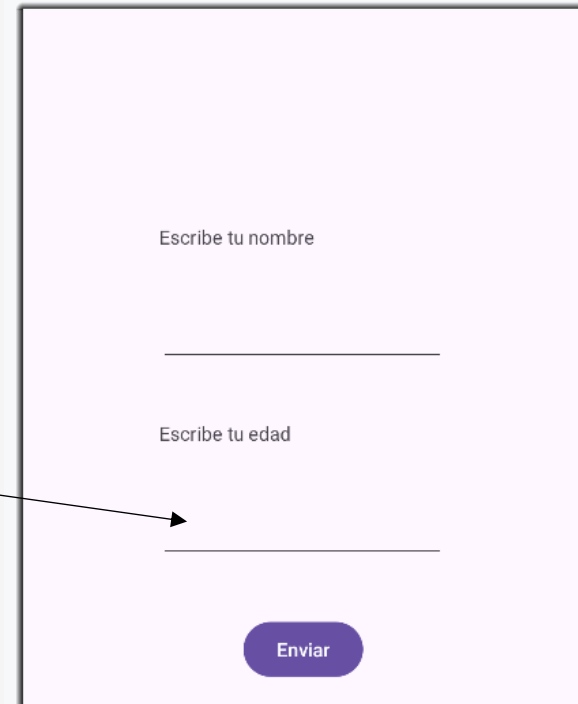
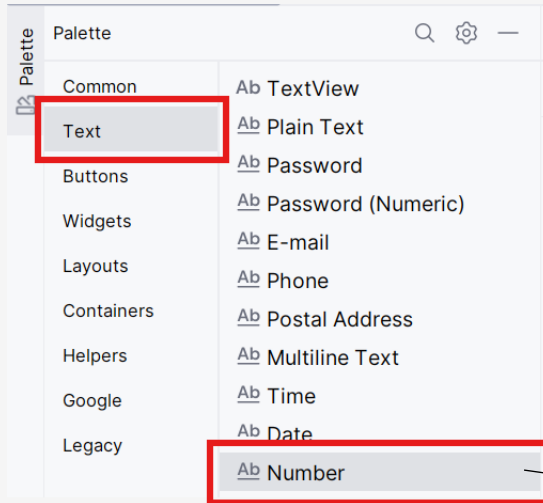
```
//Obtenemos el valor enviado por el primer Activity  
val texto = intent.getStringExtra("nombre")
```

```
//Establecemos el valor del TextView con el texto recibido  
tvNombre.text= texto
```



Pasar de un Activity a otro (intent explícito)

- Vamos a modificar MainActivity para pasar también la edad.
- Para el campo de texto de la edad elegimos el tipo Number.



Identificadores:

- **tvEdad**
- **etEdad**



Pasar de un Activity a otro (intent explícito)

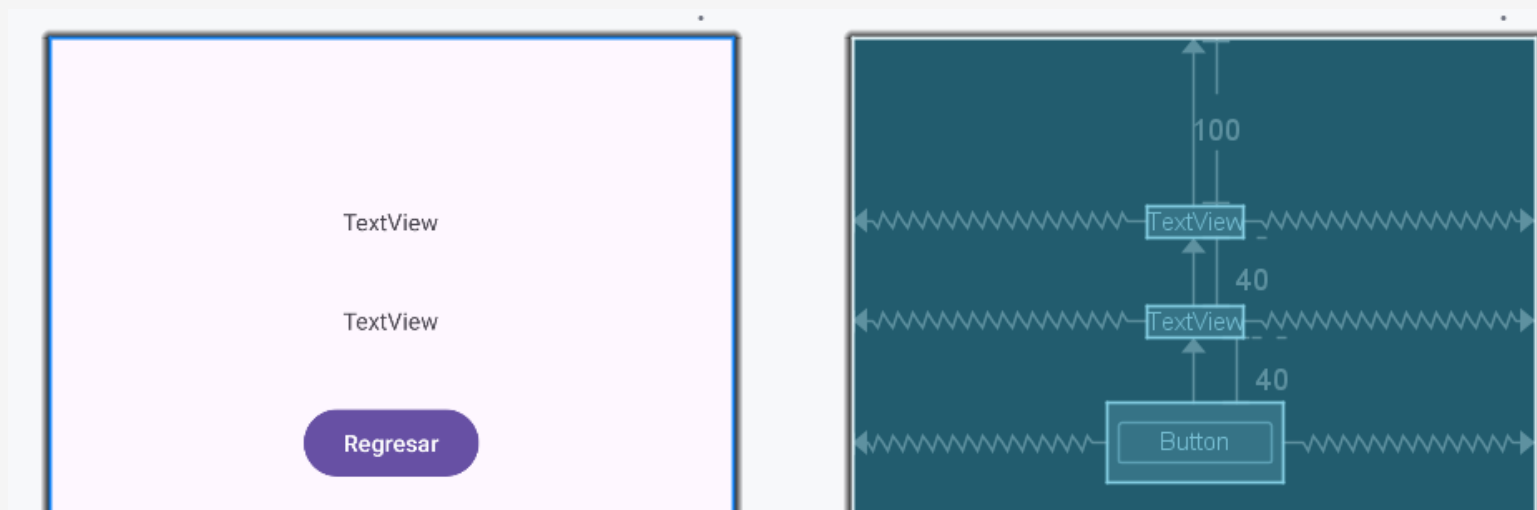
- Al poner el campo de tipo **Number** Android sólo permitirá que el contenido del campo sean números. Si pulsamos cualquier otra tecla no se añadirá al campo.
- El campo sigue siendo un **EditText** pero al seleccionar Number el atributo **android:inputType** toma el valor “**number**”

```
<EditText
    android:id="@+id/etEdad"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="100dp"
    android:layout_marginTop="40dp"
    android:ems="10"
    android:inputType="number"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/tvEdad" />
```




Pasar de un Activity a otro (intent explícito)

- Modificamos también el segundo Activity añadiendo un segundo TextView:



- Identificador segundo TextView: **tvEdadRecibida**



Pasar de un Activity a otro (intent explícito)

- Ejercicio: añade el código necesario para pasar la Edad y recibirla.
- Para pasar un entero tenemos el método:
 - `putExtra(String name, int value)`
- Y para convertir un String en un Int podemos usar:
 - `etEdad.text.toString().toInt()`
- Si el método para recibir el **nombre** es **getStringExtra()** ¿qué método crees que habrá que usar para recibir un Int?



Pasar de un Activity a otro (intent explícito)

- Así queda MainActivity

```
btEnviar.setOnClickListener {  
    val intent: Intent = Intent(this, ActivityRespuesta::class.java)  
  
    val etNombre: EditText = findViewById(R.id.etNombre)  
    intent.putExtra("nombre", etNombre.text.toString())  
  
    val etEdad: EditText = findViewById(R.id.etEdad)  
    intent.putExtra("edad", etEdad.text.toString().toInt())  
  
    startActivity(intent)  
}
```



Pasar de un Activity a otro (intent explícito)

- Así queda **ActivityRespuesta**
- El método **getIntExtra** necesita un valor por defecto, en esta caso hemos puesto 0

```
val intent=getIntent()
```

```
val tvNombre: TextView=findViewById(R.id.tvNombreRecibido)
```

```
val tvEdad: TextView=findViewById(R.id.tvEdadRecibida)
```

```
val texto = intent.getStringExtra("nombre")
```

```
val edad = intent.getIntExtra("edad",0)
```

```
tvNombre.text = texto
```

```
tvEdad.text = edad.toString()
```



Uso de un objeto Bundle

- Cuando añadimos datos con **putExtra android** crea por debajo un objeto **Bundle**.
- Si tenemos muchos datos a pasar podemos crear nosotros un objeto **Bundle** para meter los datos y luego hacer solo **putExtra** del objeto **Bundle**.

```
val intent: Intent =Intent(this, ActivityRespuesta::class.java)
```

```
val etNombre: EditText =findViewById(R.id.etNombre)
```

```
val etEdad: EditText =findViewById(R.id.etEdad)
```

```
val extras =Bundle()
```

```
extras.apply{
```

```
    putString("nombre",etNombre.text.toString())
```

```
    putInt("edad",etEdad.text.toString().toInt())
```

```
}
```

```
intent.putExtra("datos",extras)
```

```
startActivity(intent)
```



Uso de un objeto Bundle

- En el Activity que recibe el bundle, obtenemos el objeto Bundle y luego usamos métodos **getString**, **getInt** etc en vez de **getStringExtra**, **getIntExtra** etc

```
val intent=getIntent()
```

```
val extras=intent.getBundleExtra("datos")
```

```
val tvNombre: TextView=findViewById(R.id.tvNombreRecibido)
```

```
val tvEdad: TextView=findViewById(R.id.tvEdadRecibida)
```

```
val texto = extras?.getString("nombre")
```

```
val edad = extras?.getInt("edad")
```

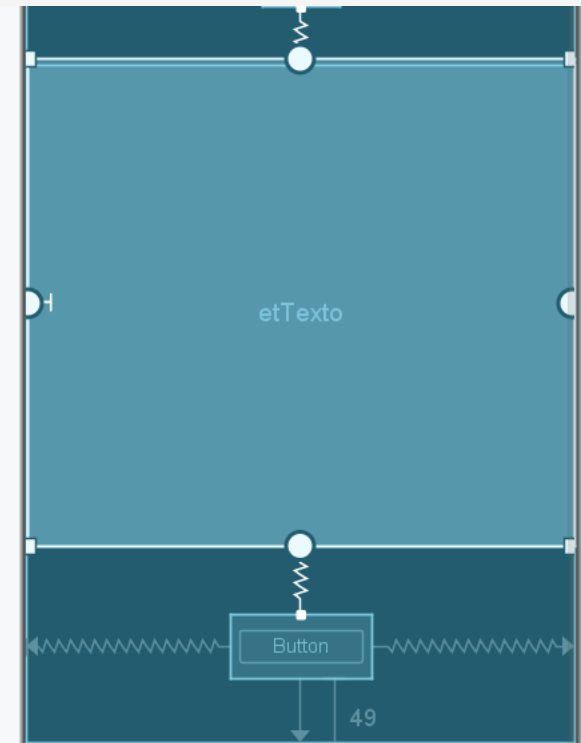
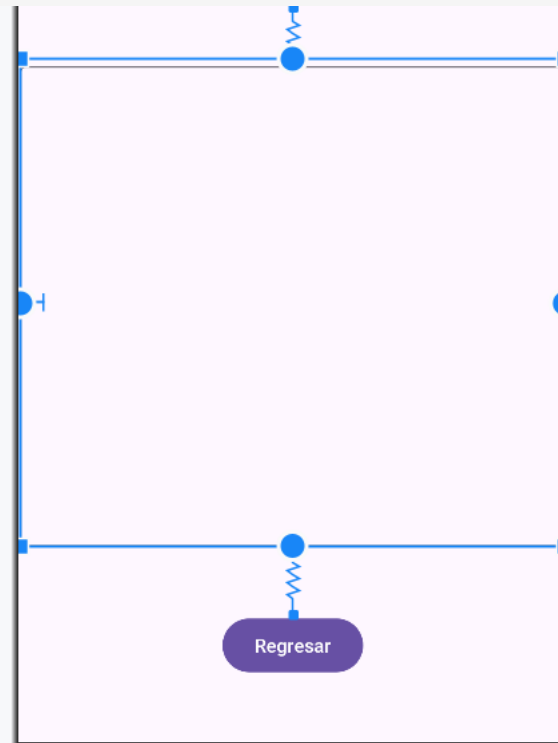
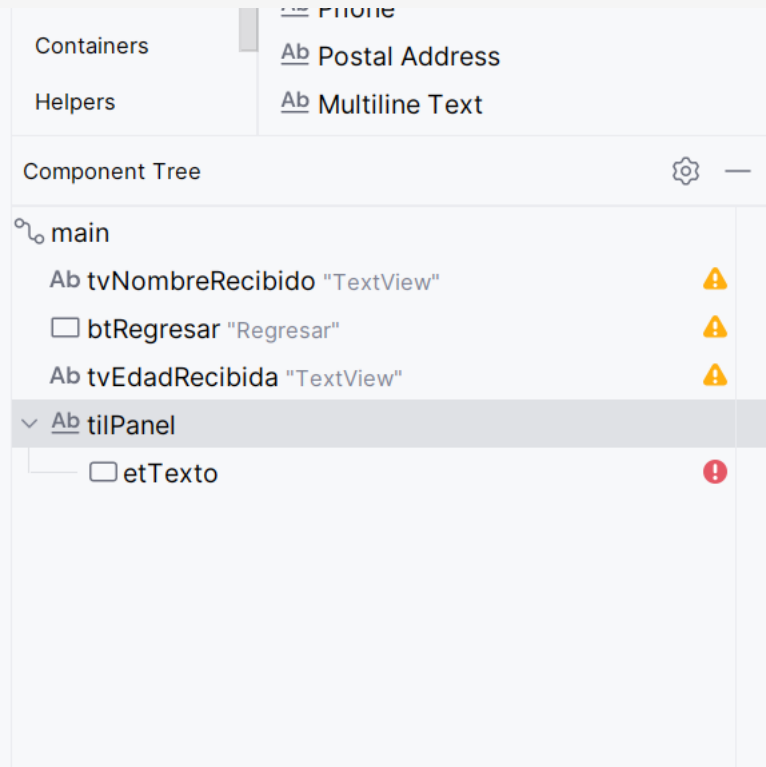
```
tvNombre.text = texto
```

```
tvEdad.text = edad.toString()
```



Ejercicio

- Vamos a modificar el diseño del activity que recibe los datos para añadir un panel que puedo contener múltiples líneas de texto.
- Añadimos un **TextInputLayout** (es sólo un panel decorativo)
- Dentro añadimos un **TextInputEditText** que es el contendrá el texto realmente





Ejercicio

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilPanel"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_marginStart="1dp"
    android:layout_marginTop="40dp"
    android:layout_marginEnd="1dp"
    android:layout_marginBottom="50dp"
    app:layout_constraintBottom_toTopOf="@+id/btRegresar"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.0"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/tvEdadRecibida"
    app:layout_constraintVertical_bias="1.0">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etTexto"
        android:gravity="top"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        />

</com.google.android.material.textfield.TextInputLayout>
```




Ejercicio

- Para escribir texto en el TextInputEditText obtendremos una referencia.
- Usaremos el método append para añadir texto.

```
val intent=getIntent()
```

```
val extras=intent.getBundleExtra("datos")
```

```
val tvNombre: TextView=findViewById(R.id.tvNombreRecibido)
```

```
val tvEdad: TextView=findViewById(R.id.tvEdadRecibida)
```

```
val etTexto: TextInputEditText=findViewById(R.id.etTexto)
```

```
val texto = extras?.getString("nombre")
```

```
val edad = extras?.getInt("edad")
```

```
tvNombre.text = texto
```

```
tvEdad.text = edad.toString()
```

```
etTexto?.append(texto+"\n")
```

```
etTexto?.append(edad.toString()+"\n")
```



Ejercicio

Escribe tu nombre

Antonio

Escribe tu edad

23

Enviar

6:29

Antonio

23

Antonio

23

Regresar



Ejercicio

- En la siguiente tabla se muestra diferentes tipos de datos que podemos pasar.
- Prueba a pasar diferentes tipos de datos y a mostrarlos en el panel de texto.

Tipo de dato	Método	Ejemplo
String	putExtra(name, String)	intent.putExtra("nombre", "Juan")
Int	putExtra(name, Int)	intent.putExtra("edad", 25)
Boolean	putExtra(name, Boolean)	intent.putExtra("activo", true)
Float	putExtra(name, Float)	intent.putExtra("precio", 9.99f)
Double	putExtra(name, Double)	intent.putExtra("nota", 8.5)
Long	putExtra(name, Long)	intent.putExtra("timestamp", System.currentTimeMillis())
Char	putExtra(name, Char)	intent.putExtra("letra", 'A')
CharSequence	putExtra(name, CharSequence)	intent.putExtra("titulo", "Hola mundo")
Array<String>	putExtra(name, Array<String>)	intent.putExtra("tags", arrayOf("A","B","C"))
IntArray	putExtra(name, IntArray)	intent.putExtra("ids", intArrayOf(1,2,3))
ArrayList<String>	putStringArrayListExtra()	intent.putStringArrayListExtra("lista", arrayListOf("Uno","Dos"))
ArrayList<Int>	putIntegerArrayListExtra()	intent.putIntegerArrayListExtra("numeros", arrayListOf(1,2,3))
Parcelable	putExtra(name, Parcelable)	intent.putExtra("user", user)
Serializable	putExtra(name, Serializable)	intent.putExtra("obj", objeto)
Bundle	putExtras(Bundle)	intent.putExtras(bundle)